

Static Trace-Capacity Accounting for Typed Agent Plans

A bits-per-session ledger for planner-style agent defenses,
with an adaptive-attacker experiment

Dibyendu Chakrabarti

Draft v1, October 2026
Technical note, not peer reviewed

Abstract

Planner-style defenses for LLM agents, such as CaMeL and FIDES, block explicit data flows but leave side channels (loop counts, induced exceptions, conditional tool calls) described only qualitatively. We give a static accountant $K(P)$ for typed plans that upper-bounds, per session, the number of attacker-distinguishable traces. We prove that an adaptive attacker running q sessions against a fixed secret learns at most $\sum_j \log_2 K_j$ bits (min-entropy leakage), show tightness on a binary-search family, and compare the bound with an exact optimal adaptive attacker on five toy channels. Under an *imposed* 0/1/3-bit per-session policy applied to the FIDES task taxonomy, oracle utility over 97 AgentDojo user tasks is 26.8%, 79.4%, and 100%, while the worst-case number of sessions to recover a 40-bit secret is unbounded, 40, and 14. The core inequality is classical (Smith; Köpf and Basin). The contribution is its use as a static, type-derived ledger for planner agents, plus a measurement of the utility cost. Scope: deterministic plans and stub extractors.

1 Motivation and gap

Planner-style defenses keep untrusted data from steering an agent’s control flow. CaMeL separates a privileged planner from a quarantined model and tracks capabilities in a custom interpreter [1]. FIDES labels data for confidentiality and integrity, enforces integrity non-interference, and settles for *explicit secrecy* on confidentiality, which still permits implicit flows through data-dependent control flow [2, 5]. Both papers state what they leave open. CaMeL describes three side channels (a loop whose iteration count depends on a private value, an induced exception, and timing) and says exploitation is hindered by bandwidth limits and attack complexity. FIDES says an observer of the tool-call sequence may infer “limited information.” Neither puts a number on it. FIDES does order output types by information capacity (a Boolean carries one bit, a string unboundedly many), but it uses that order to select policies, not to count bits.

Question. Can we compute, from types alone, a per-session bound on what an adaptive attacker learns, and compose it across sessions? The inequality is classical quantitative information flow [4, 3]. The work here is to instantiate it as a static ledger over typed plans and to measure what tightening it costs. The closest agent-side work we found is OCELOT [6], a runtime mediator that charges calibrated min-entropy costs against a per-trajectory budget; Section 6 states the difference as we read it.

2 Model

A secret S is uniform on N values and persists across sessions. In session j , plan P_j runs with an attacker-chosen untrusted input u_j , which the attacker picks as a function of earlier observations. The observer sees $\text{obs}(P, s, u)$: the sequence of attacker-visible sink calls with their visible arguments, optionally followed by a terminal marker that distinguishes normal return from uncaught exception. Let $T_u(P) = \{\text{obs}(P, s, u) : s\}$. A static accountant returns $K(P)$ with

$$|T_u(P)| \leq K(P) \quad \text{for every } u. \quad (1)$$

Table 1 gives the rules.

Plan form	Bound K
skip, halt, or any site with no secret dependence (including control dependence)	1
emit a secret-dependent value of finite type τ at a visible sink	$ \tau $ (Boolean 2, enum of k values: k)
emit an unbounded string at a visible sink	∞ (reject, or require a finite schema)
$P; Q$	$K(P) K(Q)$
if on a secret-dependent guard	$K(P) + K(Q)$
if on a guard fixed by u and public history	$\max\{K(P), K(Q)\}$
loop, secret-dependent trip count $\leq n$, body bound K_b	$\sum_{m=0}^n K_b^m$ ($= n + 1$ when $K_b = 1$)
loop, trip count fixed by u	K_b^n

Table 1: Static rules. Secret dependence would come from the confidentiality labels FIDES already propagates.

The accountant trusts three kinds of annotation: dependency flags, finite-domain declarations, and loop bounds. The code’s `validate_annotations` tests them on small domains but does not prove them.

3 Results

Theorem 1 (Soundness). *Under deterministic semantics, sound dependency labels, certified loop bounds, and context-uniform subplan bounds (each bound holds for every reachable entry state and every subset of secrets consistent with the trace so far), $|T_u(P)| \leq K(P)$ for every plan P and input u .*

Proof sketch. Structural induction. For $P; Q$, a complete trace is a prefix from P plus a residual from Q . There are at most $K(P)$ prefixes, and given any prefix, context-uniformity gives at most $K(Q)$ residuals. For a secret-dependent conditional, every trace lies in one branch’s family, so the union has at most $K(P) + K(Q)$ elements. With a guard fixed by u , one branch runs. For a loop, executions with exactly m iterations have at most K_b^m traces; take the union over m . Early termination only shortens traces. $\square$

Theorem 2 (Adaptive sessions). *Let $u_j = A_j(h_{j-1})$ for any attacker strategy and let $K_j = K(P_j)$. The transcript map $F(s) = (o_1, \dots, o_q)$ satisfies*

$$|\text{range}(F)| \leq \min \left\{ N, \prod_{j=1}^q K_j \right\}. \quad (2)$$

For uniform S , the optimal one-guess success probability is $V(S | F) = |\text{range}(F)|/N$, and the min-entropy leakage satisfies

$$L_\infty(S \rightarrow F) = \log_2 |\text{range}(F)| \leq \min \left\{ \log_2 N, \sum_{j=1}^q \log_2 K_j \right\}. \quad (3)$$

Proof sketch. A deterministic strategy is a decision tree. At depth $j - 1$ the history fixes u_j ; restricting the secret set cannot increase the number of next-session traces, so Theorem 1 caps the out-degree at K_j . A depth- q tree has at most $\prod_j K_j$ leaves, and each secret follows one path, so there are also at most N leaves. A MAP guesser is right on exactly one secret per transcript, so success is $|\text{range}(F)|/N$, and the leakage identity follows from $H_\infty(S) - H_\infty(S | F) = \log_2 (V(S | F) N)$. Because the attacker knows its own randomness, a randomized attacker’s view is the pair (R, F) ; conditioning on $R = r$ gives a deterministic tree, and averaging the Bayes vulnerabilities preserves the bound. $\square$

Corollary 1 (Ledger). *Keep, per protected secret, $B_{\text{used}} = \sum_j \log_2 K_j$ over authorized sessions, and authorize a new session only if $B_{\text{used}} + \log_2 K(P_{\text{new}}) \leq B$. Then leakage is at most B bits, provided the observation model, the secret’s identity, and the ledger persist across all counted sessions. For a 40-bit secret and $K = 2$ per session, recovery needs at least 40 sessions; with $K = 8$, at least 14.*

Theorem 3 (Tightness on a family). *For every $d \geq 1$ there is a secret domain of size $N = 2^d$, a plan with $K = 2$, and an adaptive d -session strategy whose transcript identifies the secret, so leakage equals $d = d \log_2 K$.*

Proof. The plan makes one fixed visible call exactly when $S \geq a$, where a is the attacker’s input. Binary search halves the consistent interval each session. $\square$

The product bound is attained only for full trees such as $N = 2^q$; this is a statement about a family, not about all programs.

4 Experiment: bound versus an exact optimal attacker

We implemented a small typed-plan language, the accountant, and a dynamic program that computes the maximum number of distinct transcripts over all deterministic adaptive q -session attackers. It is the strategy-tree recurrence of Köpf and Basin [3]: partition the candidate set by the observation under each input, recurse on each block, sum the leaves, and take the best input. Domains have at most 16 secrets. Theorem 1 was also checked by brute force on 3,000 random programs, and the annotation validator passes on 500 of them.

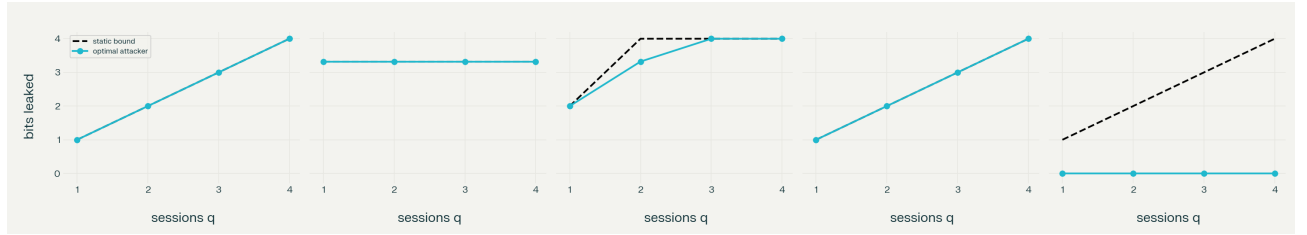


Figure 1: Leakage in bits of the optimal adaptive attacker (solid) against the static bound (dashed), for five channels.

Five channels: (1) compare: ping if $s \geq u$; (2) the loop-count channel from CaMeL’s side-channel discussion, with count = $s \leq 9$; (3) a windowed loop with count = $\min(\max(s - u, 0), 3)$; (4) an induced exception when $s \geq u$, with a visible terminal marker; (5) the same program with the marker hidden.

- Channels 1 and 4 meet the bound exactly at every q , as Theorem 3 predicts.
- Channel 2 leaks $\log_2 10 \approx 3.32$ bits in one session and then saturates, so repeating the session reveals nothing new.
- Channel 3 shows slack. After session 1 the attacker has 2 bits. At $q = 2$ the best attacker reaches 10 transcripts (3.32 bits) against a bound of 4, because the product counts combinations no adaptive attacker can realize. By $q = 3$ it reaches 4 bits = $\log_2 16$.
- Channel 5: the accountant charges 1 bit per session, but the observer sees identical traces and leakage is 0. The bound is safe and not exact, and the choice to model the terminal marker matters.

5 Utility cost under an imposed policy

FIDES classifies AgentDojo user tasks as data-independent (DI), data-independent with a constrained quarantined-LLM query (DIQ), or data-dependent (DD). Those categories do not by themselves fix bit costs. For this experiment we *impose* a policy: DI costs 0 bits (nothing in the visible trace depends on confidential data); DIQ costs 1 bit (one Boolean query is the only secret-dependent influence); DD costs 3 bits (all secret-dependent planning goes through one enum of at most eight outcomes). A per-session budget B admits every category with cost $\leq B$. Task counts are the FIDES Table 2 percentages times suite sizes of 40, 20, 16, and 21 (sizes inferred from the task-index ranges; they sum to the 97 user tasks the paper reports).

Budget	Workspace	Travel	Banking	Slack	All 97
0 bits (DI)	47.5%	0.0%	18.8%	19.0%	26.8%
1 bit (+DIQ)	95.0%	90.0%	56.2%	57.1%	79.4%
3 bits (+DD)	100%	100%	100%	100%	100%

Table 2: Oracle utility: share of tasks allowed. The 100% row is an upper bound under the imposed interface, not a claim that DD tasks stay solvable.

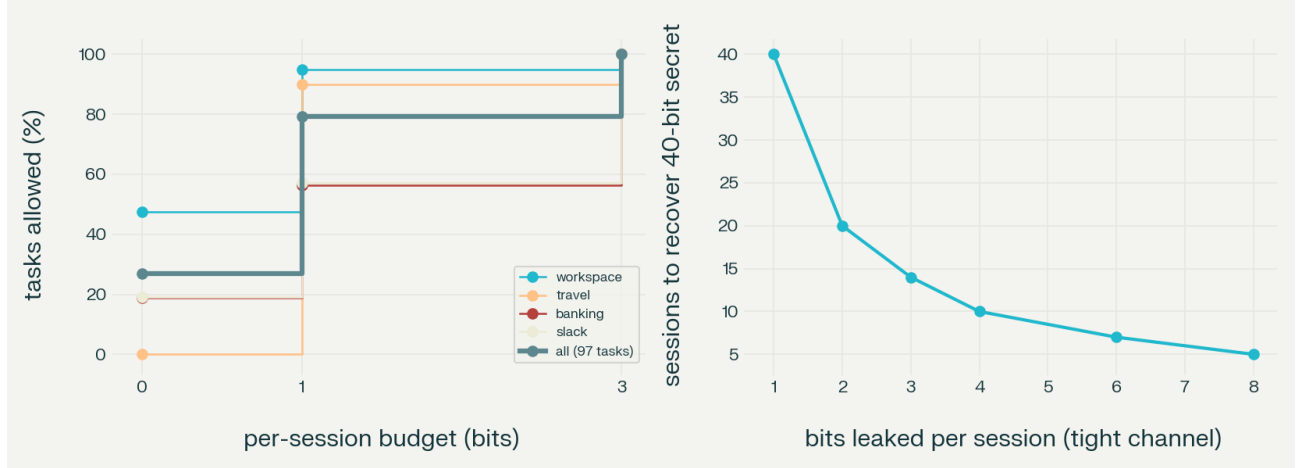


Figure 2: Left: utility steps by budget. Right: worst-case sessions to recover a 40-bit secret over a tight channel (unbounded at 0 bits).

Moving from 1 to 3 bits per session raises oracle utility from 79.4% to 100% and cuts the worst-case sessions to compromise from 40 to 14. These are worst-case figures for a tight channel and a fixed secret; real plans usually leak less.

6 Related work and limitations

Related work. Smith showed that for a deterministic program under a uniform prior, min-entropy leakage equals the log of the number of distinct outputs [4]. Köpf and Basin modeled adaptive side-channel attackers as strategy trees and bounded their information gain by the number of measurements [3]; our Theorem 2 is that argument applied to typed plans. Explicit secrecy [5], which FIDES builds on, defines the attacker’s knowledge set whose size we count. OCELOT [6] charges calibrated min-entropy costs at runtime against a per-trajectory ledger, reports leakage in bits against task success, and treats persistent memory as a channel. As we read its Sections 3.2 to 4.4, its deductive budget is stated at trajectory scope, while ours fixes one secret across adaptively chosen sessions and computes costs statically from types. We have not verified every detail of that comparison.

Limitations.

- Deterministic plans and stub extractors. Real LLM extractors are stochastic; a noisy extension needs its own theorem. A binary symmetric channel with flip probability p carries at most $1 - H_2(p)$ bits per use even with feedback, but that is a Shannon bound, not a min-entropy bound, and should not be dropped into the ledger unproved.
- Annotations are trusted. Soundness assumes the dependency flags, finite-domain declarations, and loop bounds are correct; the validator tests them only on small domains.
- No mutable cross-session agent memory. Sessions share the secret and the attacker’s inputs only, so memory-write channels are out of scope.

- Reachability slack. Products and sums can count transcript combinations that cannot co-occur (Figure 1, channel 3), and repeated or correlated extractors are charged again.
- Toy scale. At most 16 secrets, and the optimal-attacker program reuses one plan across sessions, although the theorem allows plans to vary.
- The 0/1/3-bit costs are an imposed policy, and utility is an oracle upper bound.
- A per-secret ledger needs an operational notion of “the same secret” (rotation, multiple accounts).
- Nothing here is a claim about CaMeL or FIDES as shipped.

7 Companion: measuring risk when no bound exists

Some risk cannot be bounded this way, because it is empirical: how often an attack succeeds against a model. Best-of- N jailbreaking found that negative-log attack success follows a power law in the number of samples [7], and Schaeffer et al. explain such aggregate power laws through heavy-tailed distributions of per-problem success probability [8]. A simple model captures this. Let a task’s single-attempt success probability be $p \sim \text{Beta}(a, b)$ across tasks, with independent attempts. Population attack success after N attempts per task is

$$\text{ASR}(N) = 1 - \frac{B(a, b + N)}{B(a, b)}, \quad 1 - \text{ASR}(N) \approx \frac{\Gamma(a + b)}{\Gamma(b)} N^{-a}.$$

The tail exponent a governs how fast risk grows.

Take two defenses with the same mean single-attempt rate of 2%: A with $\text{Beta}(2, 98)$ and B with $\text{Beta}(0.2, 9.8)$. Their $\text{ASR}(1)$ is identical, and their growth is not.

Attempts N	1	10	100	1,000	10,000
A: exponent 2	2.0%	17.6%	75.4%	99.2%	100%
B: exponent 0.2	2.0%	13.5%	38.8%	60.7%	75.2%

Table 3: Population attack success (exact, from the Beta mixture).

A small red-team can estimate the exponent. We simulated 300 tasks with 20 attempts each, 100 replications, and fit a beta-binomial by maximum likelihood. The median estimate of a is 2.10 for A (5th to 95th percentile 1.09 to 20.2, so the light-tail case is poorly identified) and 0.20 for B (0.14 to 0.31). For B, the fitted model predicts $\text{ASR}(10,000) = 0.751$ (5th to 95th percentile 0.649 to 0.869) against a true 0.752. A homogeneous baseline that assumes every task has the pooled success rate predicts 100% for B at $N = 1,000$, against a truth of 60.7%. A single pooled ASR can badly overstate risk for a heavy-tailed defense, and the exponent is the number worth reporting.

Caveat. This models independent sampling attacks. Optimization-based adaptive attacks can break tasks that sampling never would; a published study broke eight indirect-prompt-injection defenses with adaptive attacks at over 50% success [9]. These curves are therefore not upper bounds on risk from an optimizing attacker.

Reproducibility

Repository `leakage_lab`. Setup: `pip install matplotlib pytest scipy numpy`. Run `pytest -q tests` (10 tests), then `python experiments.py` (Figure 1, `results.csv`), `python utility.py` (Figure 2), and `python heavy_tail.py` (Figure 3, fixed seed 2026).

References

- [1] E. Debenedetti et al. Defeating Prompt Injections by Design (CaMeL). arXiv:2503.18813, 2025.

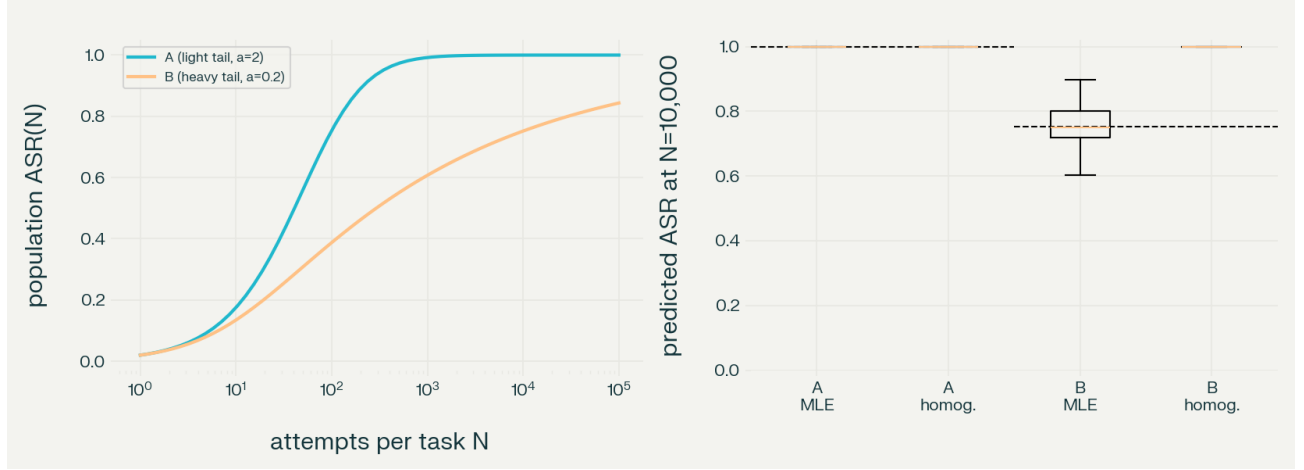


Figure 3: Left: population ASR for the two defenses. Right: predicted ASR at $N = 10,000$ from a small red-team; dashes mark the truth.

- [2] M. Costa et al. Securing AI Agents with Information-Flow Control (FIDES). arXiv:2505.23643, 2025.
- [3] B. Köpf and D. Basin. An information-theoretic model for adaptive side-channel attacks. ACM CCS, 2007.
- [4] G. Smith. On the foundations of quantitative information flow. FoSSaCS, 2009.
- [5] D. Schoepe, M. Balliu, B. C. Pierce, and A. Sabelfeld. Explicit secrecy: a policy for taint tracking. IEEE EuroS&P, 2016.
- [6] OCELOT: Inference-Leakage Budgets for Privacy-Preserving LLM Agents. arXiv:2606.12341, 2026.
- [7] J. Hughes et al. Best-of-N Jailbreaking. 2024.
- [8] R. Schaeffer et al. How do large language monkeys get their power (laws)? ICML, 2025.
- [9] Adaptive Attacks Break Defenses Against Indirect Prompt Injection Attacks on LLM Agents. NAACL Findings, 2025 (arXiv:2503.00061).